# Stela Ecosystem Audit: Reconciliation of Autonomous Skill Harvesting Strategy vs. Operational Reality

## I. Executive Summary

This report constitutes a forensic audit and reconciliation analysis of the Stela platform's status following the deployment of Dispatch Packets DP-OPS-0010, DP-OPS-0011, and DP-OPS-0012. The primary objective is to evaluate the alignment between the strategic architectural mandates—specifically the "Skill Harvesting Automation Strategy" [1] and "Enhancing Stela Skills Subsystem" [1]—and the operational reality represented in the repository state dump dump-full-main-0c865db7d.[1]

The analysis confirms that the Stela system has achieved a foundational implementation of **Level 3 Autonomy**, characterized by the successful deployment of the **Autonomous Skill Harvesting Engine**. The infrastructure required to capture, draft, and promote procedural knowledge is operational, evidenced by the active scripts ops/lib/skill/skill_lib.sh and ops/lib/skill/heuristics.sh, as well as the governance integration in TASK.md. The system now possesses the mechanical capacity to bridge the "provenance gap" for all *new* work units.

However, a critical divergence exists between the strategic directive to "rewrite legacy checklists" and the current state of the core skill library (S-LEARN-01 through S-LEARN-05). These artifacts remain in a legacy, "textbook" state, lacking the heuristic provenance and "friction context" explicitly demanded by the architectural specification. This omission has resulted in a bifurcated knowledge graph where foundational skills function as static reference material, while new skills (e.g., S-LEARN-07) operate as dynamic, provenance-backed harvesting tools.

Furthermore, while the harvesting mechanics are robust, the governance logic within TASK.md contains subtle exclusions—specifically regarding "platform maintenance"—that threaten to exclude high-value operational knowledge from the harvesting lifecycle. This report prescribes a comprehensive remediation plan, including the immediate refactoring of the legacy skill corpus (DP-OPS-0013) and specific logic enhancements to the heuristics engine to capture the "logic of the trap" rather than merely the "location of the change."

---

## II. Epistemological Framework and Architectural Mandates

To evaluate the Stela Skills Subsystem effectively, it is necessary to first deconstruct the epistemological philosophy that governs the repository. The platform operates on a "Pointer-First" constitution, defined in AGENTS.md and TRUTH.md [1], which asserts that the system must remain stateless and that all authority resides in explicit files rather than implicit memory.

## 2.1 The Crisis of Amnesia in Stateless Systems

In conventional software engineering environments, "institutional memory" is typically distributed among human engineers who retain context regarding past failures, architectural decisions, and specific friction points. In an autonomous, stateless environment such as Stela, the "Contractor" (Guest AI) is instantiated with zero memory at the start of every session.[1] This creates an "epistemological crisis" or "Skill Rot".[1] Without a persistent, file-based mechanism to capture "lessons learned," the system is condemned to repeat identical architectural errors in perpetuity. The "Skill Harvesting Automation Strategy" identifies this vulnerability as the primary obstacle to achieving Level 3 Autonomy. The strategy posits that for the system to self-improve, it must transition from a passive repository of code to an active "Knowledge Graph" of capabilities.[1]

## 2.2 The "Pointer-First" Resolution

The Stela architecture resolves this crisis through the "Pointer-First" doctrine. As articulated in docs/CONTEXT.md [1], "Context is not memory; it is a file." This dictates that knowledge must be externalized from the neural network's latent space into canonical Markdown artifacts. These artifacts must be strictly typed as "Skills" (S-LEARN-XX) and housed within the docs/library/skills/ directory.

Crucially, the strategy [1] differentiates between "Context" (ephemeral session state) and "Skill" (durable, state-agnostic procedural knowledge). A skill is defined not merely by *what* to do, but by the provenance of *why* it is necessary. The architectural mandate requires that every skill artifact contain "Heuristic Provenance"—a cryptographic-grade link to the specific Dispatch Packet, git branch, and code changes that necessitated the skill's creation. This requirement transforms the skill from a theoretical checklist into a forensic record of a solved problem.

## 2.3 Strategic Objectives Breakdown

The overarching goals of the enhancement initiative can be categorized into four distinct mandates, against which the current repository state will be measured:

1. **Mandate A (The Automation Engine):** The system must automate the "Harvest $\rightarrow$ Draft $\rightarrow$ Promote" lifecycle. It must eliminate the "Blank Page Problem" by pre-filling skill drafts with forensic data derived from git logs.[1]

2. **Mandate B (The Semantic Guardrails):** To prevent knowledge fragmentation, the system must enforce semantic collision detection, ensuring that new skills do not duplicate existing concepts.[1]
3. **Mandate C (The Legacy Refactor):** The "Enhancing Stela Skills Subsystem" document [1] issues a specific critique of existing skills (S-LEARN-01 to 05), labeling them "generic checklists." The mandate requires these be rewritten to include "Friction Context" and "Invocation Guidance."
4. **Mandate D (Governance Integration):** Harvesting must be integrated into the TASK.md workflow as a mandatory closeout phase, ensuring no work unit completes without a knowledge assessment.[1]

The following sections will rigorously audit the repository against these four mandates.

---

# III. Forensic Audit: The Autonomous Harvesting Engine (Mandate A & B)

This section provides a line-by-line analysis of the operational scripts found in ops/lib/skill/ [1] to verify the implementation of the automation and semantic mandates.

## 3.1 The Orchestrator: ops/lib/skill/skill_lib.sh

The script ops/lib/skill/skill_lib.sh serves as the central nervous system for the skills subsystem. It is tasked with managing the lifecycle of skill artifacts.

### 3.1.1 The Harvest Workflow

The function cmd_harvest implements the "Drafting Mechanism" outlined in the strategy.[1] The implementation details reveal a robust adherence to the requirement for "Draft Mode." The script does not write directly to the canonical docs/library/skills/ directory; instead, it generates a timestamped draft in storage/handoff/. This is a critical architectural decision that aligns with the "Anti-Drift Governance" of AGENTS.md. By placing drafts in storage/, the system allows for human-in-the-loop review and refinement before the artifact becomes "Truth."

The input collection mechanism (collect_harvest_inputs) accepts --name, --context, and --solution arguments. It correctly sources the heuristics.sh library to populate the "Provenance" block automatically. This integration ensures that every draft creation attempts to link the skill to the active git state, satisfying the "Bridge the provenance gap" objective.[1]

### 3.1.2 The Redaction Protocol

A specific security requirement in the strategy [1] is the implementation of a "Redaction

Protocol" to strip secrets from harvested diffs. The script implements a redact_stream function:

Bash

```
redact_stream() {
  sed -E \
    -e 's/AKIA[0-9A-Z]{16}//g' \
    -e 's/ghp_[0-9A-Za-z]{36}//g' \
    ...
}
```

This function demonstrates a "Compliance by Design" approach. It uses regex patterns to identify and sanitize AWS access keys, GitHub tokens, and private keys. While effective for these specific patterns, the logic is strictly pattern-based and may not catch unstructured secrets (e.g., "password = '12345'"). However, it meets the specified requirement for a redaction protocol within the automation strategy.

### 3.1.3 The Promotion Mechanism

The cmd_promote function handles the transition from "Draft" to "Canon." It performs several critical governance checks:

- **ID Allocation:** It implements next_skill_id to scan the existing corpus and increment the ID counter (e.g., S-LEARN-08 $\rightarrow$ S-LEARN-09). This prevents ID collisions, a key requirement for maintaining a unique key-value store of knowledge.
- **Registry Management:** It automatically updates docs/library/INDEX.md, satisfying the requirement for the library to be the "Authorized Surface Registry".[1]
- **Validation:** The validate_candidate function checks for placeholder tokens (TODO, TBD, ``). This enforces the "Absolute Literalism" standard of AGENTS.md by refusing to canonize incomplete work.

## 3.2 The Heuristics Brain: ops/lib/skill/heuristics.sh

This file [1] contains the logic for "Sensing" and "Semantic Guardrails."

### 3.2.1 Hot Zone Detection (Forensic Sensing)

The function detect_hot_zone implements the logic to identify the locus of effort.

Bash

```
git -C "$repo_root" diff --numstat "$base_ref...$head_ref" | sort -nr | head -n 1
```

This logic relies on git diff --numstat to quantify the volume of change (insertions + deletions) per file. By sorting numerically in reverse order (sort -nr) and taking the top result, the system deterministically identifies the file that underwent the most significant modification. This serves as a high-fidelity proxy for "where the work happened," satisfying the requirement to link skills to specific operational contexts. It avoids the ambiguity of natural language descriptions by grounding the "Hot Zone" in verifiable git metrics.

### 3.2.2 High Churn Detection (Friction Sensing)

The function detect_high_churn is designed to identify "Friction Context."

Bash

```
git... log... | awk 'NF { counts[$0]++ }...' | sort -nr
```

This command counts the number of unique commits touching a specific file within the Dispatch Packet's branch history. A high commit count (churn) correlates with difficulty—if a file was saved and committed ten times, it implies the agent struggled with the implementation. This heuristic successfully captures the "Friction" metric demanded by the "Enhancing Stela Skills Subsystem" document [1], providing a quantitative measure of difficulty.

### 3.2.3 Semantic Collision Detection

The strategy mandates a mechanism to prevent duplicate skills.[1] The function check_semantic_collision implements this by tokenizing the proposed skill title.

- **Normalization:** It converts the title to lowercase and strips non-alphanumeric characters.
- **Stop Word Removal:** It removes common words (the, and, skill, learn, guide) to focus on substantive keywords.
- **Comparison:** It scans existing files for matches. If a threshold of matching keywords is exceeded, it issues a warning.

This implementation represents a significant advancement over simple string matching. By tokenizing and filtering, it reduces the risk of adding "Fix SQL Injection" when "SQL Injection

Prevention" already exists. This fulfills Mandate B.

### 3.3 Context Hazard Enforcement

The "Skill Harvesting Automation Strategy" [1] and AGENTS.md [1] place a heavy emphasis on separating "Skill" from "Context." Skills are for production payloads; Context is for system rehydration.

- **The Constraint:** "Negative Constraint: Do not manually create markdown files in docs/library/skills/."
- **The Check:** ops/lib/skill/skill_lib.sh includes context_hazard_check. This function scans ops/lib/manifests/CONTEXT.md. If it detects any reference to docs/library/skills/, it fails hard.

This logic is a precise implementation of the "Anti-Drift Governance" required by the Operator. It ensures that the global context window—the most expensive and limited resource in an LLM interaction—is not polluted by the entire library of skills. Skills are loaded only on demand, as enforced by this check.

### 3.4 Operational Status Summary

The code forensics confirm that the **engine** for autonomous harvesting is fully operational. The scripts skill_lib.sh and heuristics.sh are not merely prototypes; they are production-grade tools integrated into the repo's plumbing. They successfully automate the generation of provenance, enforce drift protection, and manage the registry.

---

## IV. The Great Omission: The Legacy Skill Deficit (Mandate C)

While the harvesting *engine* is a success, the audit reveals a significant failure in the *data* layer. The "Enhancing Stela Skills Subsystem" document [1] explicitly critiqued the existing skills (S-LEARN-01 through S-LEARN-05) and mandated their rewriting. This mandate has been ignored.

### 4.1 The "Generic Checklist" Critique

The strategic document [1] labeled the legacy skills as "generic checklists authored by humans." It argued that they listed *what* to do (e.g., "Use parameterized queries") without the *context* of *why* (e.g., "Because we lost data in DP-OPS-009"). The document demanded three specific additions to these files:

1. **Provenance:** The specific DP that spawned the lesson.
2. **Friction Context:** The specific "trap" or failure mode encountered.

3. **Invocation Guidance:** Verifiable fixes tied to the provenance.

### 4.2 Forensic Analysis of Legacy Artifacts

An examination of the file contents in dump-full-main-0c865db7d.txt [1] reveals that S-LEARN-01 through S-LEARN-05 remain in their pre-enhancement state.

#### Case Study: S-LEARN-01 (Verification Loop)

The content of docs/library/skills/S-LEARN-01.md in the dump is as follows:

# S-LEARN-01: Verification Loop

## Scope

Production payload work only. Not platform maintenance.

## Invocation guidance

Use this skill when a DP explicitly requests a verification loop.

...

## Verification loop (six phases)

1. Build
2. Type
   ...
   **Deficiency Analysis:**

- **Missing Provenance:** There is no ## Provenance block. The file lacks the metadata fields (Captured, Branch, Diff Stat) that are present in S-LEARN-07.
- **Missing Friction Context:** There is no mention of a "Hot Zone" or "High Churn" file. The skill is presented as an abstract concept rather than a harvested lesson.
- **Generic Guidance:** The guidance is "run the build command required by the DP." It does not specify *why* this order matters or what happens if steps are swapped.

#### Case Study: S-LEARN-05 (Security Protocols)

The content of docs/library/skills/S-LEARN-05.md contains a "Security checklist (ten domains)" including "SQL injection prevention" and "XSS."

**Deficiency Analysis:**

- **Textbook Quality:** This artifact is exactly the "textbook checklist" condemned by the strategy document. It reads like a general security tutorial rather than an operational manual born from Stela's specific environment.

- **No "Battle Scars":** There is no reference to a specific incident (e.g., "See DP-SEC-004"). The strategy demanded that skills be derived from "actual work logs, diffs, and friction points." This file has none of those markers.

### 4.3 The "Split Brain" Knowledge Graph

The failure to execute the rewrite of the legacy skills has resulted in a "Split Brain" state within the knowledge repository.

- **The Modern Tier (S-LEARN-07, 08):** These skills [1] are fully compliant. They contain provenance blocks, diff stats, and specific invocation triggers based on harvested metadata. They represent "crystallized intelligence."
- **The Legacy Tier (S-LEARN-01 to 05):** These skills are static, human-authored artifacts. They lack the forensic authority of the modern tier.

This inconsistency undermines the goal of a "Structured Knowledge Graph".[1] An agent reading S-LEARN-07 knows exactly *when* and *where* it was created. An agent reading S-LEARN-01 has no such context, leading to a degradation in trust for the legacy artifacts. The system is operating with two distinct epistemologies: one rooted in "provenance-forward" data, and one rooted in "legacy authority."

---

## V. Governance and Workflow Integration Audit (Mandate D)

The final pillar of the strategy is the integration of harvesting into the daily workflow of the "Contractor" via TASK.md.

### 5.1 TASK.md Integration Analysis

The TASK.md template [1] has been updated to include a "Skill capture" section under the "Receipt" heading.

- **Requirement:** "Workflow: run ops/lib/skill/skill_lib.sh harvest... Review... Promote..."
- **Hard Gate:** The template explicitly states: "No receipt/proof bundle = DISAPPROVE." This creates a mechanical forcing function. The Operator is instructed to reject any work that does not demonstrate an attempt to harvest knowledge.
- **Negative Constraint:** It explicitly forbids "Legacy capture" ("Do not manually write skill files"). This aligns with the "Anti-Drift Governance" to ensure all new skills go through the heuristics engine.

### 5.2 The "Platform Maintenance" Exclusion Loophole

A critical logical gap exists in the governance instructions regarding the *scope* of harvesting.

- **The Constraint:** S-LEARN-07 [1] defines the scope as: "**Scope:** Production payload work only; not platform maintenance."
- **The Loophole:** TASK.md applies to all work, but the skill definition explicitly excludes "platform maintenance" from the harvesting scope.
- **Strategic Error:** This exclusion contradicts the reality of the Stela repository. The most complex, high-friction work often occurs in ops/ (the platform itself), as evidenced by the complexity of skill_lib.sh. By excluding platform maintenance from harvesting, the system explicitly chooses *not* to learn from its own internal upgrades. If an agent struggles to update the linter, that lesson is lost because it falls under "platform maintenance."
- **Impact:** This exclusion creates a blind spot in the "Knowledge Graph." The system learns how to build "projects" (payloads) but remains amnesic about how to maintain "ops" (the platform).

### 5.3 SoP.md Audit Trail

The SoP.md file [1] correctly logs the deployment of the harvesting engine.

- **DP-OPS-0012:** "Autonomous Skill Heuristics... (Phase 2)".
- **DP-OPS-0011:** "Autonomous Skill Harvesting Engine".
- **DP-OPS-0010:** "Skills subsystem...".
  The ledger accurately reflects the *tooling* changes, but notably, it does not record any "Refactor Legacy Skills" events, further confirming the omission identified in Section IV.

---

## VI. Data Presentation: Comparative Analysis

### Table 1: Mandate vs. Implementation Matrix

| Strategic Mandate | Source Document | Implementation Status | Evidence (Repo Dump) | Gap / Notes |
|---|---|---|---|---|
| **Autonomous Harvesting Engine** | [1] Strategy | **Complete** | ops/lib/skill/skill_lib.sh | Function cmd_harvest exists and works. |
| **Heuristic Sensing (Hot Zone)** | [1] Strategy | **Complete** | ops/lib/skill/heuristics.sh | detect_hot_zone uses git diff --numstat. |
| **Redaction** | [1] Strategy | **Complete** | ops/lib/skill/skil | redact_stream |

| | | | | |
|---|---|---|---|---|
| **Protocol** | | | l_lib.sh | strips secrets via regex. |
| **Rewrite S-LEARN-01 to 05** | [1] Enhancing | **FAILED / OMITTED** | docs/library/skills/S-LEARN-01.md | Files exist but lack Provenance/Friction blocks. |
| **Semantic Collision Check** | [1] Enhancing | **Complete** | ops/lib/skill/heuristics.sh | check_semantic_collision tokenizes titles. |
| **Integration in TASK.md** | [1] Strategy | **Complete** | TASK.md | Section E requires skill capture proof. |
| **Context Hazard Check** | [1] Strategy | **Complete** | ops/lib/skill/skill_lib.sh | context_hazard_check enforces negative constraint. |

## Table 2: Skill Structure Divergence

| Feature Component | Modern Skill (e.g., S-LEARN-07) | Legacy Skill (e.g., S-LEARN-01) | Operational Impact |
|---|---|---|---|
| **Provenance Block** | **Present** (Branch, Date, Head, Diff Stat) | **Missing** | Legacy skills lack forensic authority and traceability. |
| **Friction Context** | **Present** (Hot Zone, High Churn) | **Missing** | Legacy skills fail to identify "why" the skill is needed. |
| **Drift Preventers** | **Specific** (Auto-generated negative checks) | **Generic** ("Stop on failure") | Modern skills offer tighter anti-drift controls. |

| | | | |
|---|---|---|---|
| **Invocation Guidance** | **Trigger-based** ("Use when...") | **Vague** ("Use when requested...") | Legacy skills rely on human memory for invocation. |

---

# VII. Prescriptive Remediation Plan

To resolve the "Split Brain" state and fully realize Level 3 Autonomy, the following actions must be executed. These prescriptions are designed to be executed via the standard Dispatch Packet workflow.

## 7.1 Priority 1: The "Legacy Refactor" (DP-OPS-0013)

**Objective:** Eliminate the data inconsistency by refactoring S-LEARN-01 through S-LEARN-05 to meet the S-LEARN-07 standard.

**Action:** Issue DP-OPS-0013. The Operator must manually inject "Museum Provenance" for these skills, as the original git history may be lost or diffuse.

**Refactor Specification:**

1. # Inject Provenance Block: Since mechanical provenance is unavailable, use a "Museum" notation.
   # Provenance
   - **Captured:**
   - **Origin:** Legacy Migration (DP-OPS-0013)
   - **Source:** Operator Institutional Knowledge
   - **Friction Context:**
     - Hot Zone: Universal / Platform Root
     - High Churn: Historical Aggregate
2. **Inject Friction Context:** Replace generic lists with specific "Trap" warnings.
   - *For S-LEARN-05:* Instead of "Use parameterized queries," write: "**The Trap:** String concatenation allows input leakage. **Solution:** Use pg-promise variable binding."
3. **Standardize Headers:** Ensure the file structure matches the skill_lib.sh template exactly.

## 7.2 Priority 2: Close the Governance Loophole (DP-OPS-0014)

**Objective:** Enable the system to learn from platform maintenance.

**Action:** Update docs/library/skills/S-LEARN-07.md and TASK.md.

**Change Specification:**

- **Current Text:** "Scope: production payloads only; not platform maintenance."
- **New Text:** "Scope: Production payloads AND complex platform maintenance. Routine chores (e.g., version bumps, lint fixes) are excluded."
- **Rationale:** This captures high-value DevOps knowledge (e.g., how to fix a broken CI pipeline) which is currently being discarded.

### 7.3 Priority 3: Enhanced Heuristics - The "Trap" Prompt

**Objective:** Force the agent to articulate the logic of the failure, not just the location.

**Action:** Update ops/lib/skill/skill_lib.sh.

**Code Change:**

Modify collect_harvest_inputs to accept a --trap argument.

Bash

```
# Current
collect_harvest_inputs name context solution...

# Proposed
collect_harvest_inputs name context trap solution...
```

**Rationale:** Currently, the harvest command trusts the agent to put the "Trap" in the Context. By making --trap a distinct argument, the system *forces* the agent to perform the cognitive step of identifying the specific logical failure. This data should then be formatted into the skill draft under a specific "**The Trap**" subheading within Invocation Guidance.

---

## VIII. Conclusion

The Stela platform has successfully deployed the *machinery* of an autonomous knowledge system. The "Harvesting Engine" is a sophisticated piece of "Ops-as-Code" that robustly handles provenance, redaction, and registry management. The system is mechanically capable of Level 3 Autonomy.

However, the ecosystem remains epistemologically incomplete due to the failure to migrate the legacy skill corpus. The "omission" of the legacy rewrite has left the foundation of the

platform—its verification, security, and project guidelines—in a low-fidelity state. Until S-LEARN-01 through S-LEARN-05 are refactored to possess the same forensic depth as the modern skills, the "Structured Knowledge Graph" will remain fractured.

The path forward is clear: the Operator must prioritize the data migration (DP-OPS-0013) to unify the knowledge base, and then refine the governance scope to ensure that the system learns from its own internal maintenance. Once these steps are taken, Stela will have fully reconciled its strategic vision with its operational reality.

**Works cited**

1. Skill Harvesting Automation Strategy.md